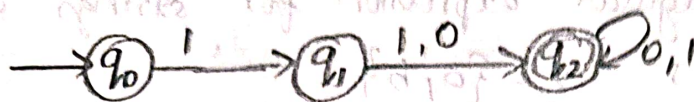


- 1) Draw transition diagram for NFA (with or without ϵ -moves) for strings starting with '10' or '11'. $\Sigma = \{0, 1\}$.

→



- 2) Define the language acceptability by DFA, NFA and ENFA.

→ DFA

: A string is accepted by a DFA if, starting from the initial state and processing the input string one symbol at a time, the automaton ends in one of the accepted states.

: For each input symbol, there is exactly one state to which the automaton can transition.

NFA

: A string is accepted by an NFA if there exists at least one sequence of transitions, starting from the initial state, that leads to an accept state after processing the input string.

: This allows for multiple paths through the automaton for a single input.

ϵ -NFA

: A string is accepted by ϵ NFA if there exists at least one sequence of transitions (including those that may involve ϵ transitions) that leads from the initial state to an accept state after processing the input string.

: This adds another layer of non-determinism since the automaton can "move" without reading input.

3) Write down the regular expression for strings with even number of a's $\Sigma = \{a, b\}$.

→ The regular expression is

$$\Rightarrow (b^* ab^* ab^*)^*$$

4) How can we identify two equivalent states in a DFA?

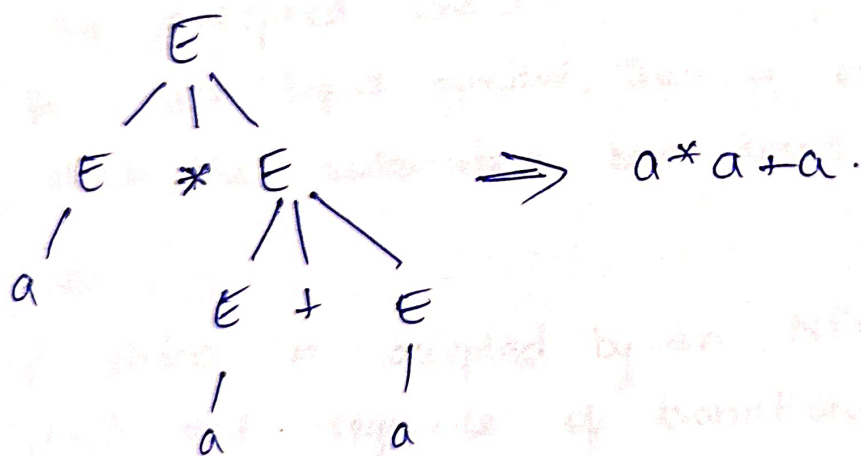
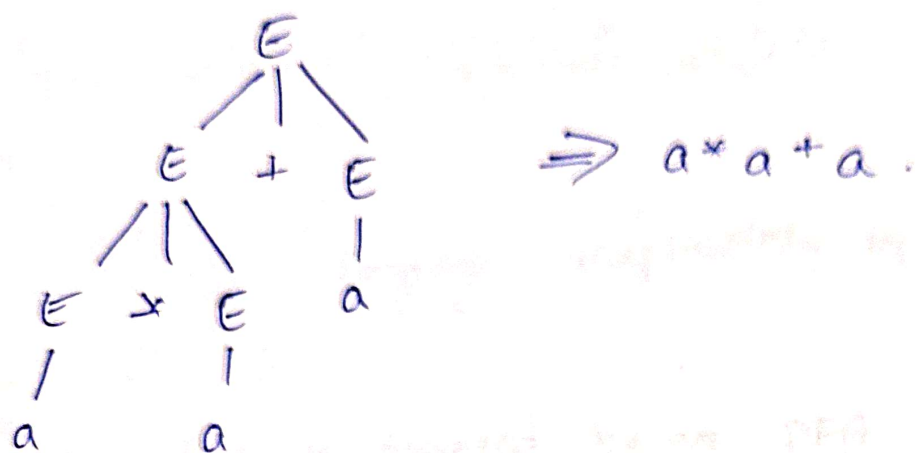
→ Two states in a DFA are equivalent if they have the same output for every possible input, and their next states are also equivalent. This will be written as $p \equiv q$, where p & q are two states.

* We can use minimization method or table filling method to find the equivalence.

- Create a table that looks at pairs of states and determines which state must be equivalent if the pair is to be equivalent.

* Two DFA's are equivalent if they have the same alphabet and accept the same set of strings.

5) Is the grammar $E \rightarrow E+E/E \times E/a$ ambiguous? Justify your answer.



$\Rightarrow$ It has two parse tree $\Rightarrow$ Grammar is ambiguous.

6. State Myhill-Nerode Theorem.

Myhill Theorem states that the following statements are equivalent.

- (i) Let the set $L \subseteq \Sigma^*$ is accepted by some finite state automata.
- (ii) L is the union of some of the equivalent classes of right invariant equivalence relation of finite index.
- (iii) Let the equivalence relation R_L be defined by $x R_L y$ iff $\forall z \in \Sigma^*$ xz is in L then yz is also in L & R_L has finite index.

7. Differentiate between DPDA and NPDA.

Non deterministic PDA

- * More powerful than DPDA
- * It is not possible to convert every NPDA to DPDA.
- * The language accepted by NPDA is not a subset of language accepted by DPDA.
- * The language accepted by NPDA is called Non deterministic CFL.
- * For every input with current state we can have multiple moves.

Deterministic PDA.

- * less powerful than NPDA.
- * It is possible to convert every DPDA to a corresponding NPDA.
- * The language accepted by DPDA is a subset of language accepted by NPDA.
- * The language accepted by DPDA is called deterministic CFL.
- * For every input with the current state there is only one move.

list out the transition rules ' δ ' used to convert given CFG G to its equivalent NPDA M .

There are two rules:

- * for a non-terminal x have the productions $x \rightarrow y/z$ then
 $\delta(q, \epsilon, x) = \{(q, y), (q, z)\}$
- * for each terminal in CFG $\delta(q, a, a) = (q, \epsilon)$

Explain how Turing Machine differs from PDA.

PDA

- Processes input string left to right
- Only memory access is the top of the stack.
- Non-deterministic
- Halts when all input is read
- No "blank" symbol

Turing Machine

- Processes input string in any order; it starts out on the tape.
- Can move tape head to any point on the tape.
- Deterministic.
- Halts whenever $q_{\text{-accept}}$ or $q_{\text{-reject}}$ is reached.
- "Blank" symbol included.

Why TM model is widely accepted and considered as a general model for computers?

Turing machines provide a simple yet powerful model of computation that has been widely used in research and has had a profound impact on our understanding of algorithms and computation.

11 a) Prove that for every NFA M , there exist an equivalent DFA M'

Induction hypothesis.

Consider a string y where $|y| = n$ and theorem proved for y

$$\delta'(q_0, y) = [p_0, p_1, \dots, p_j] \text{ iff}$$

$$\delta(q_0, y) = [p_0, p_1, \dots, p_j]$$

Induction step

Consider a string x , where $|x| = n+1$

$$\begin{aligned} 1^{\text{st}} \text{ part} &= \delta'(q_0, x) = \delta'(q_0, y) = \delta'(\delta'(q_0, y), a) \\ &= \delta'([p_0, p_1, \dots, p_j], a) \\ &= [q_0, q_1, \dots, q_k] \end{aligned}$$

$$\begin{aligned} 2^{\text{nd}} \text{ part} &= \delta(q_0, x) = \delta(q_0, y_0) = \delta(\delta(q_0, y), a) \\ &= \delta([p_0, p_1, \dots, p_j], a) \\ &= \{q_0, q_1, \dots, q_k\} \end{aligned}$$

Since theorem is proved for base case, n and $n+1$
The theorem is proved for all values of ' n '. i.e. there
for all strings x

So for every NFA there exists an equivalent DFA

Equivalence of DFA and NFA

→ For every NFA 'm' there exists an equivalent DFA 'm' which simulates the behaviour of m

→ If L is a language accepted by an NFA then there exists a DFA, which also accepts L

→ NFAs and DFAs are equivalent

→ Any NFA can be converted into its equivalent DFA

PROOF: Let $M(Q, \Sigma, \delta, q_0, F)$ be an NFA accepting L

Then construct its equivalent DFA $M'(Q', \Sigma, \delta', q_0', F')$

Where $Q' = 2^Q$

$q_0' = [q_0]$

$F' =$ All subsets of Q which contains an element of F

$$\begin{aligned}\delta'([q_0, q_1, \dots, q_i], a) &= \delta(\{q_0, q_1, \dots, q_i\}, a) \rightarrow (1) \\ &= \delta(q_0, a) \cup \delta(q_1, a) \dots \cup \delta(q_i, a)\end{aligned}$$

By definition

$$\delta'([q_0, p_1, \dots, p_i], a) = [q_0, q_1, \dots, q_k] \text{ iff}$$

$$\delta([q_0, p_1, \dots, p_i], a) = [q_0, q_1, \dots, q_k] \rightarrow (2)$$

We want to prove that $\delta'(q_0', x) = [q_0, q_1, \dots, q_i]$ iff

$$\delta(q_0, x) = \{q_0, q_1, \dots, q_i\}$$

Consider the string x and apply mathematical induction on |x|

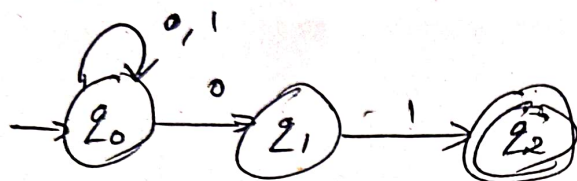
Base case: |x| = 0, $x = \epsilon$

1st part: $\delta'(q_0', \epsilon) = q_0' = [q_0]$

2nd part: $\delta(q_0, \epsilon) = \{q_0\}$ Theorem proved for base case.

1b) Design an NFA (without ϵ -moves) for strings ending with '01'. Convert it into equivalent DFA.
 $\Sigma = \{0, 1\}$

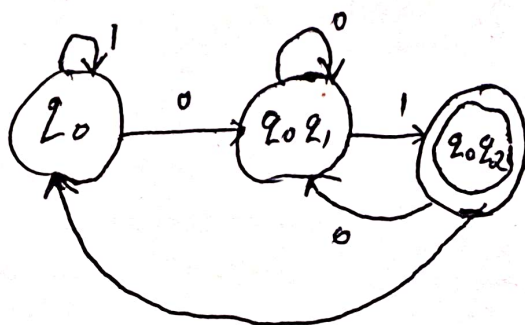
String end with 01



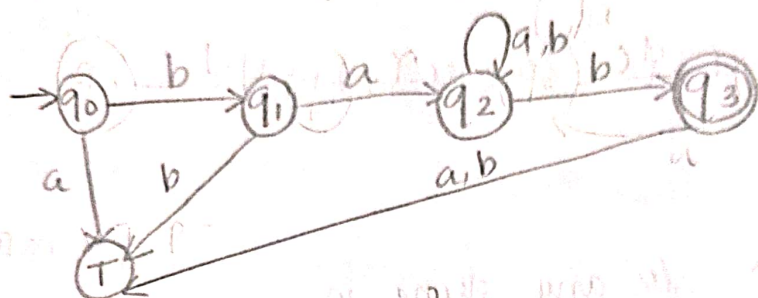
Q/ Σ	0	1
q ₀	{q ₀ , q ₁ }	q ₀
q ₁	ϕ	q ₂
q ₂	ϕ	ϕ

DFA

Q/ Σ	0	1
q ₀	{q ₀ , q ₁ }	q ₀
{q ₀ , q ₁ }	{q ₀ , q ₁ }	{q ₀ , q ₂ }
{q ₀ , q ₂ }	{q ₀ , q ₁ }	q ₀



- 12 a) Draw the transition diagram of DFA for $L = \{baxb/x \in \{a,b\}^*\}$ and obtain its regular grammar.



$$G = VTPS$$

$$V = \{q_0, q_1, q_2, q_3, T\}$$

$$T = \{a, b\}$$

$$S = \{q_0\}$$

$$P: q_0 = bq_1$$

$$q_0 = aT$$

$$q_1 = aq_2$$

$$q_1 = bT$$

$$q_2 = aq_0$$

$$q_2 = bq_2$$

$$q_2 = bq_3$$

$$q_3 = aT$$

$$q_3 = bT$$

2. Prove that, if L is accepted by an ordinary NFA, there exist an equivalent equivalent ϵ -NFA that also accepts L .

$$M = (Q, \Sigma, \delta, q_0, F) \rightarrow \text{NFA}$$

$$M = (Q, \Sigma, \delta', q_0, F') \rightarrow \text{NFA}$$

$$\text{where } F' = \begin{cases} F \cup \{q_0\} & \text{if } \epsilon\text{-closure}(q_0) \text{ contains state in } F \\ F & \text{otherwise} \end{cases}$$

$$\delta' = \hat{\delta}(q, a)$$

$$= q \in Q \text{ and } a \in \Sigma$$

$$\delta'(q, a) = \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(q), a))$$

By induction,

$$\delta'(q_0, w) = \hat{\delta}(q_0, w) \text{ for any string } w$$

For $w = \epsilon$,

$$\delta'(q_0, \epsilon) \neq \hat{\delta}(q_0, \epsilon) \text{ since } \hat{\delta}(q_0, \epsilon) = \{q_0\}.$$

$$\delta'(q_0, \epsilon) = \epsilon\text{-closure}(q_0)$$

So induction is not true for string having length zero.

$\therefore$ if L is accepted by an ordinary NFA, there exist an equivalent ϵ -NFA that also accepts L .

13. Obtain the regular expression equivalent to the following DFA?



- b. Develop equivalent automata for the Regular expression $(ab + b)^* (a + bb)^* a^*$

Answers

a)

$$q_0 = \epsilon + q_0 a \rightarrow \textcircled{1}$$

$$q_1 = q_0 b + q_1 b \rightarrow \textcircled{2}$$

$$q_2 = q_1 a + q_2 a + q_2 b \rightarrow \textcircled{3}$$

We have two final states.

$$q_0 ; \textcircled{1} \Rightarrow q_0 = \epsilon + q_0 a$$

$$q_0 = \epsilon \cdot a^*$$

$$R = Q + RP$$

$$\Rightarrow R = QP^*$$

$$\underline{q_0 = a^*} \rightarrow \textcircled{4}$$

$$q_1 ; \textcircled{2} \Rightarrow q_1 = q_0 b + q_1 b$$

$$q_1 = a^* b + q_1 b$$

$$q_1 = a^* b (b)^*$$

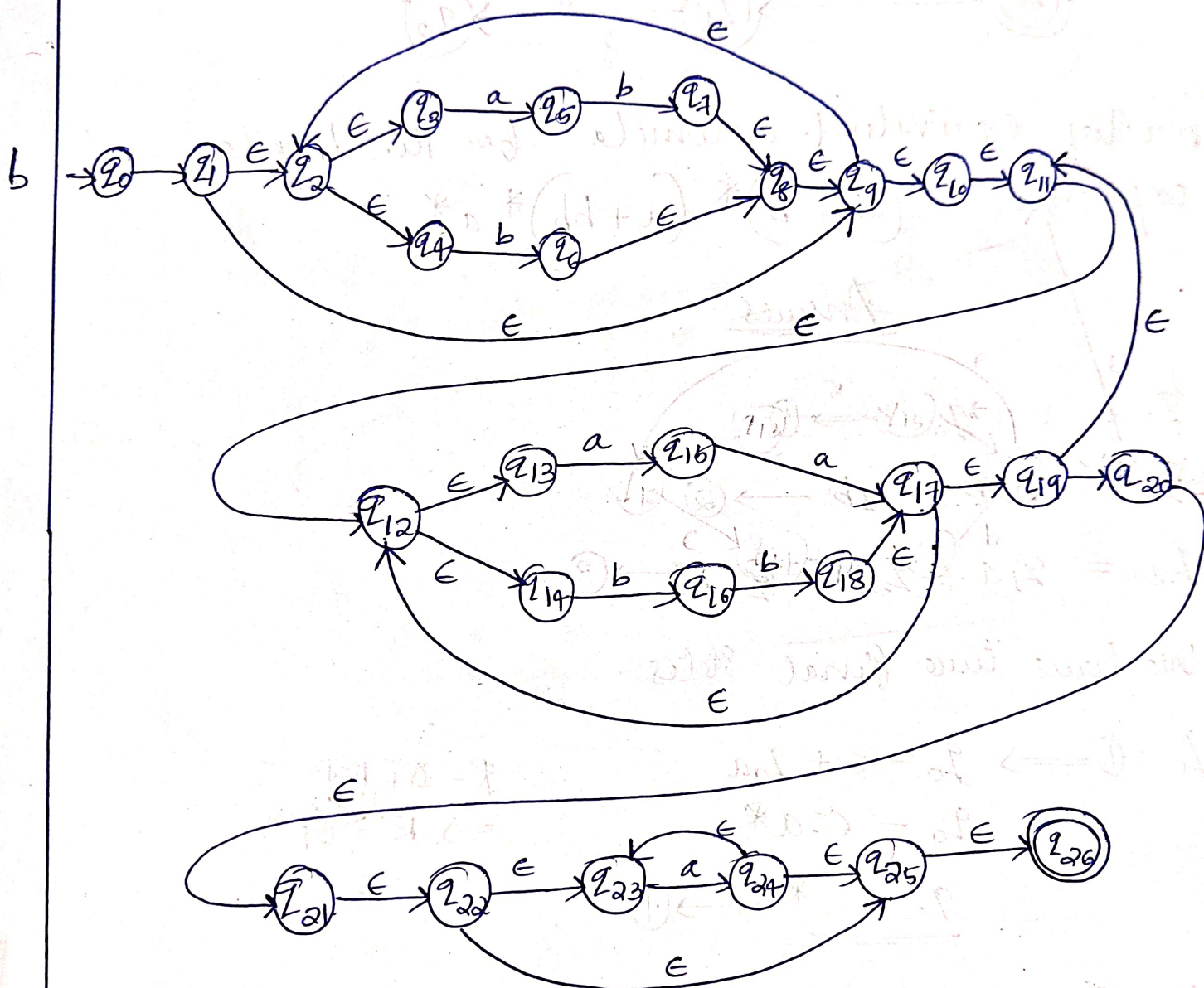
from $\textcircled{4}$

Regular expression = $a^* + a^*b(b)^*$

$$= a^*(\epsilon + bb^*)$$

$$\epsilon + RR^* = R^*$$

$$= \underline{\underline{a^*b^*}}$$



14

Q Prove that for every regular expression 'R', there is an ENFA 'M'.

Q Using pumping lemma, is $L = \{a^p / p \text{ is prime no}\}$ is not regular?

Ans

Q

$$L = \{a^p / p \text{ is prime no}\}$$

$$L = \{aa, aaaa, aaaaaa, \dots\}$$

$$w = \underbrace{aaaaaa}_2 \underbrace{aa}_4 \underbrace{aa}_2$$

$$p = 8$$

$$(i) |y| > 0$$

$$y = aa$$

$$|y| = 2$$

$$2 > 0 \text{ true}$$

$$(ii) |xy| \leq p$$

$$xy = aaaaaa$$

$$|xy| = 6$$

$$6 \leq p \text{ true}$$

$$(iii) xy^2z$$

$$i = 2$$

$$\underbrace{aaaa}_2 \underbrace{[aa]}_4 \underbrace{aa}_2$$

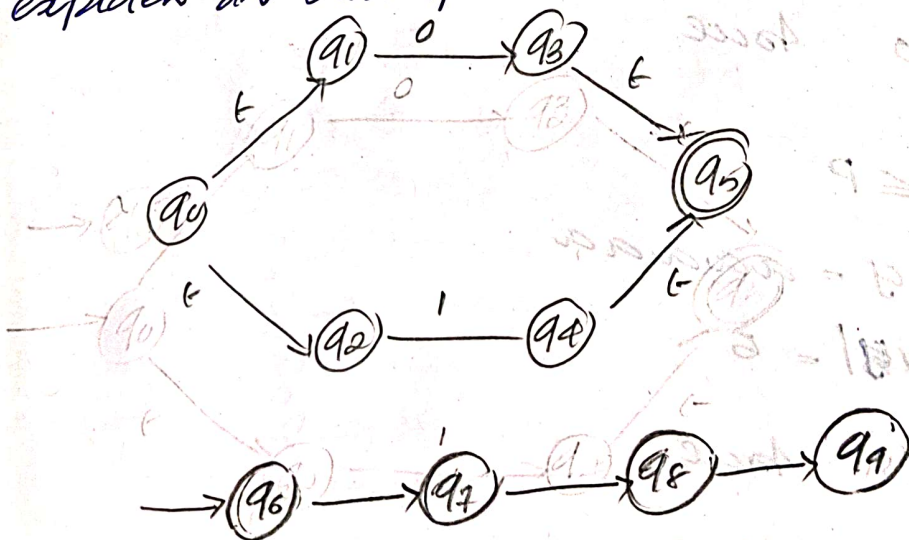
$$aaaaaaaaaa = a$$

1 is not a prime number

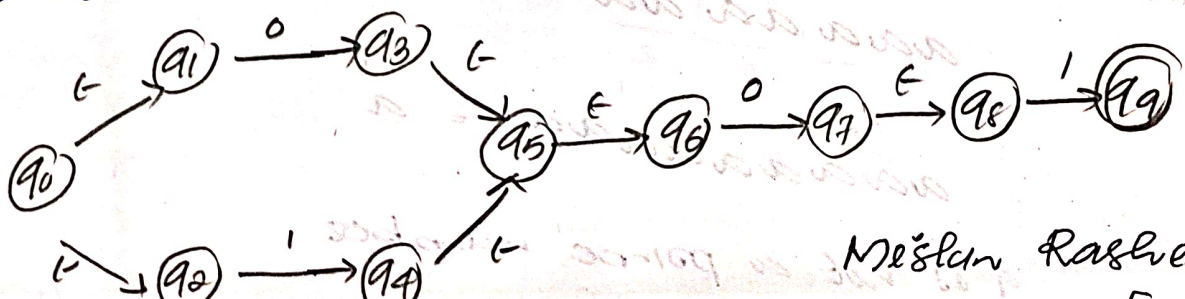
① For every regular expression R , there exists an NFA 'M' that can be constructed using a systematic approach. We construct NFAs for the basic regular expressions such as the empty set, the empty string ϵ and a single character a using simple state diagrams.

For complex expressions, we build ϵ -NFAs by combining smaller ones using operations like union, concatenation and closure stars. This step by step construction ensures that for any regular expression A , there is an equivalent NFA M that accepts the same language.

We can explain an example here
 $[0+1]^*01$



combining both we get



Muhammad Raheem

15 a) What is Greibach Normal Form (GNF)?
Convert the following CFG to GNF

$$S \rightarrow ABA$$

$$A \rightarrow aA/\epsilon$$

$$B \rightarrow bB/\epsilon$$

→ A grammar is said to be in GNF if every production is of the form $A \rightarrow a\alpha$ where A is non-terminal and a is a terminal and α is the string of non-terminals or ϵ

$$S \rightarrow ABA$$

$$A \rightarrow aA/\epsilon$$

$$B \rightarrow bB/\epsilon$$

Step 1:-

$$- S \rightarrow ABA$$

$$A \rightarrow \epsilon$$

$$B \rightarrow \epsilon$$

$$S \rightarrow BA$$

$$S \rightarrow AA$$

$$S \rightarrow B$$

$$S \rightarrow A$$

$$S \rightarrow \epsilon$$

(Substituting without ϵ values).

Removing ϵ ,

$$S \rightarrow ABA|BA|AA|B|A$$

$$A \rightarrow aA/a$$

$$B \rightarrow bB/b$$

Step 2:- Remove unit productions.

$$- S \rightarrow ABA \mid BA \mid AA \mid B \mid A$$

$$A \rightarrow aA \mid a$$

$$B \rightarrow bB \mid b$$

$$- S \rightarrow ABA \mid BA \mid AA \mid a \mid \cancel{aA} \mid bB \mid b$$

$$A \rightarrow aA \mid a$$

$$B \rightarrow bB \mid b$$

Step 3:- Remove left recursion.

There is no left recursion.

Step 4:- Perform substitution to convert to GNF.

$$- S \rightarrow ABA$$

$$S \rightarrow \cancel{aA} \cancel{aA} \cancel{bB} \quad aBC \quad (\because A \rightarrow a)$$

$$* S \rightarrow aABC \quad (\because A \rightarrow aA)$$

$$- S \rightarrow A \cancel{A}$$

$$S \rightarrow aA \cancel{A} \mid a \cancel{A} \quad (\because A \rightarrow aA \mid a)$$

$$- S \rightarrow BA$$

$$S \rightarrow bBA \mid bA \quad (\because B \rightarrow bB \mid b)$$

Final

$$S \rightarrow aABC \mid aBC \mid aAA \mid aA \mid bBA \mid bA \mid a \mid aA \mid bB \mid b$$

$$A \rightarrow aA \mid a$$

$$B \rightarrow bB \mid b$$

b) Design CFG for the following languages.

(i) $L = \{w w^R / w \in \{a, b\}^*\}$ where w^R represents reverse of w

$$w = \{e, abcb, aabbcbba, \dots\}$$

$$G = (V, T, P, S)$$

$$S \rightarrow aSa | bSb | e$$

$$V = S$$

$$T = abc$$

$$P = S \rightarrow aSa | bSb | e$$

$$S \rightarrow S$$

ii) String with exactly two zeroes

$$w = \{00, 100, 001, 1001, \dots\}$$

$$S \rightarrow SOS | 1 | e$$

$$V = S$$

$$T = 0, 1$$

$$P = S \rightarrow SOS | 1 | e$$

$$S = S$$

iii) $(011+1)^* (01)^*$

$$w = \{e, 1, 011, 01, \dots\}$$

$$S \rightarrow e | 1S | 01S$$

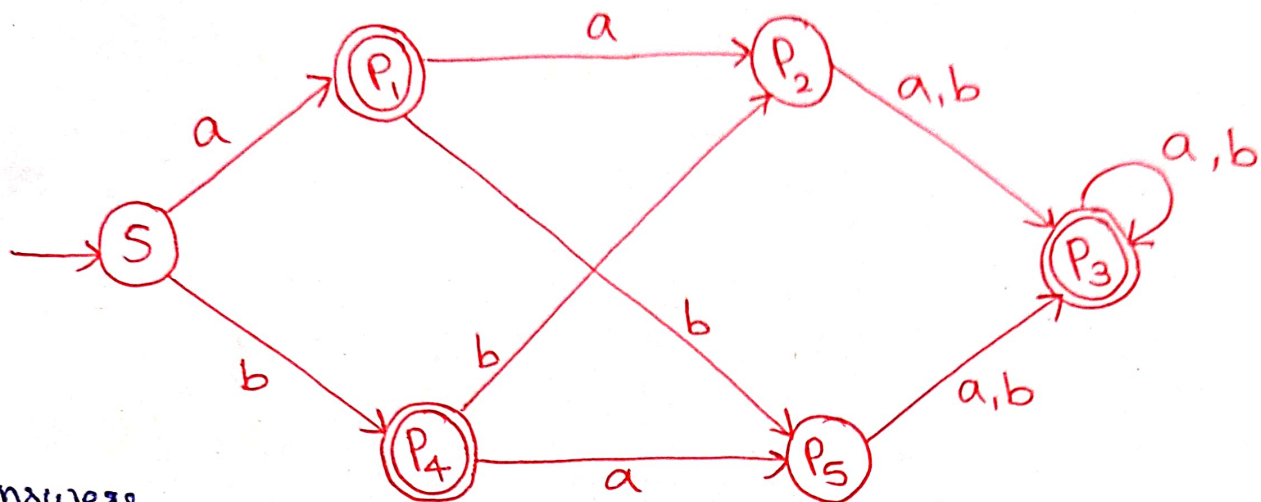
$$V \rightarrow S$$

$$T = 0, 1$$

$$P = S \rightarrow e | 1S | 01S$$

$$S \rightarrow S$$

16) (a) Minimize the following DFA using Myhill-Nerode theorem



Answer

	S	P ₁	P ₂	P ₃	P ₄	P ₅
S	X	X	X	X	X	X
P ₁	✓	X	X	X	X	X
P ₂	✓	✓	X	X	X	X
P ₃	✓	✓	✓	X	X	X
P ₄	✓		✓	✓	X	X
P ₅	✓	✓		✓	✓	X

(S, P₂)

$$\delta(S, a) = P_1 \quad \delta(S, b) = P_4$$

$$\delta(P_2, a) = P_3 \quad \delta(P_2, b) = P_3$$

(P₁, P₃)

(Marked)

$$\delta(P_1, a) = P_2$$

$$\delta(P_3, a) = P_3$$

(P_1, P_4)

$$\delta(P_1, a) = P_2 \quad \delta(P_1, b) = P_5$$

$$\delta(P_4, a) = P_5 \quad \delta(P_4, b) = P_2$$

(P_3, P_4)

$$\delta(P_3, a) = P_3 \quad (\text{Marked})$$

$$\delta(P_4, a) = P_5$$

(S, P_5)

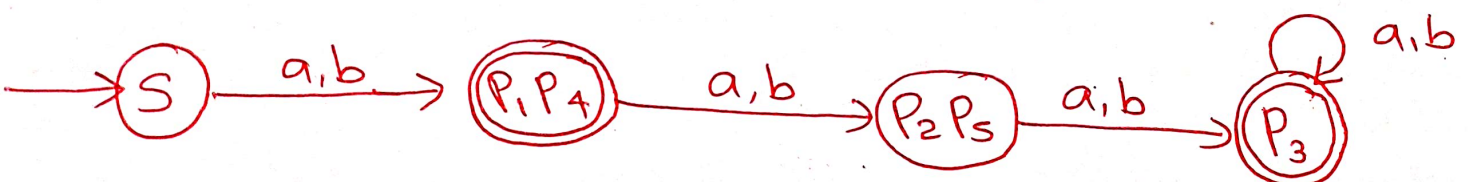
$$\delta(P_5, a) = P_3 \quad (\text{Marked})$$

$$\delta(S, a) = P_1$$

(P_2, P_5)

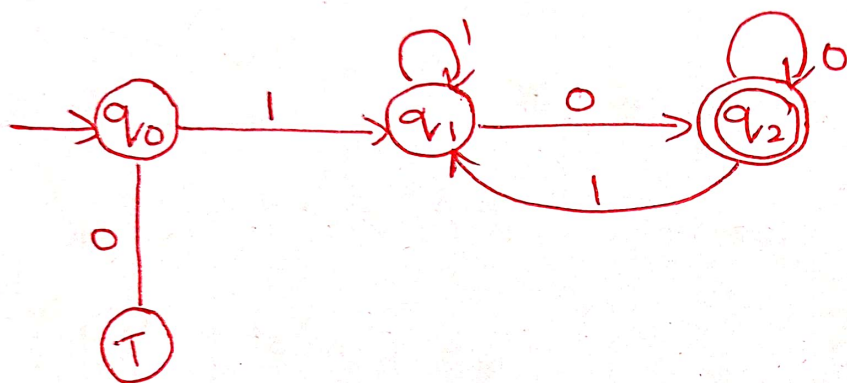
$$\delta(P_2, a) = P_3 \quad \delta(P_2, b) = P_3$$

$$\delta(P_5, a) = P_3 \quad \delta(P_5, b) = P_3$$



- 16) (b) Show the equivalence classes of Canonical Myhill-Nerode relation for the languages of binary strings with odd number of zeros and even number of ones.

Answer.



Equivalence classes

$$C_1 = \{x \in \Sigma^* / \delta(q_0, 0) = \delta(q_0, 1) = q_0\}$$

$$= \{ \epsilon \}$$

$$C_2 = \{x \in \Sigma^* / \delta(q_0, 0) = \delta(q_0, 1) = q_1\}$$

$$= \{1, 11, 111, 101, 1011, \dots\}$$

$$C_3 = \{x \in \Sigma^* / \delta(q_0, 0) = \delta(q_0, 1) = q_2\}$$

$$= \{10, 1010, 100, \dots\}$$

1. a) Prove that for every PDA accepted by empty stack, there exist an equivalent PDA accepted by final state. (7M)

A) Part-1

Let $M_1 \rightarrow$ PDA accepted by final state

$M_2 \rightarrow$ PDA accepted by empty stack

We need to prove that any language accepted by M_1 can be accepted by M_2 i.e., $L(M_1) = L(M_2)$.

Tuples of M_1 and M_2 :

$$M_1 = (Q, \Sigma, \delta_1, q_0, z_0, \Gamma, F)$$

$$M_2 = (Q \cup \{q_0', q_e\}, \Sigma, \delta_2, q_0', z_0', (\Gamma \cup z_0'), \emptyset)$$

Transition functions:

$$\textcircled{1} \delta_2(q_0', \epsilon, z_0') = (q_0, z_0 z_0')$$

$\hookrightarrow$ no i/p

$$\textcircled{2} \delta_2(q_i, a, z) = \delta_1(q_i, a, z)$$

$\downarrow$
i/p

$$\textcircled{3} \delta_2(q_i, \epsilon, z) = (q_e, \epsilon)$$

$$\textcircled{4} \delta_2(q_e, \epsilon, z) = (q_e, \epsilon)$$

$$\therefore \textcircled{1} (q_0', x, z_0') \vdash (q_0, x, z_0 z_0')$$

ω , initially, whatever the i/p symbol, just push z_0 to top of the stack.

$$\textcircled{ii} (q_0, x, z_0 z_0') \vdash (q_f, \epsilon, z_0 z_0')$$

$$\textcircled{iii} (q_0', x, z_0) \vdash (q_e, \epsilon, \epsilon)$$

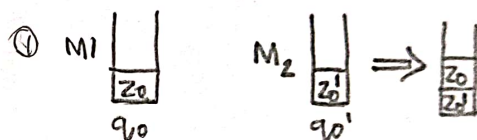
$\therefore M_1$ reaches final state, M_2 also reaches its stack as above.

(Final state not required as its empty stack)

$q_0' \rightarrow$ starting state of M_2

$q_e \rightarrow$ new arising state

$z_0 \rightarrow$ bottom of M_2



$\textcircled{2}$ Whatever transition is done in M_1 is also done in M_2 .

$\textcircled{3}$ If M_1 reaches final state, M_2 goes to arising state q_e with no i/p i.e., ϵ .

$\textcircled{4}$ When q_e and no i/p & whatever in stack, it remains in q_e and will keep on popping stack content. So stack is finally empty - push z_0 to top of the stack.

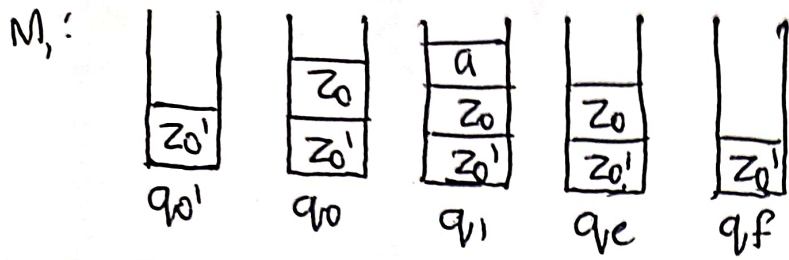
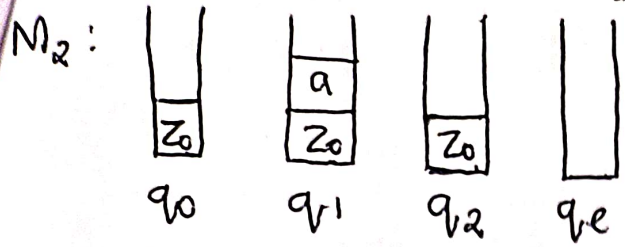
From z_0 , it is same as M_1 and finally reaches final state of M_1 and i/p is completely consumed, top of stack is $z_0 z_0'$. M_2 reaches q_e and i/p will be completely consumed and stack is empty.

Part-2

If M_2 is accepting by empty stack, then M_1 should be accepting same string by final state.

$$M_1 = (Q \cup \{q_0', q_f\}, \Sigma, \delta, q_0', z_0', (\Gamma \cup z_0), \{q_f\})$$

$$M_2 = (Q, \Sigma, \delta_2, q_0, z_0, \Gamma, \emptyset)$$



- ① $\delta_1(q_0', \epsilon, z_0') = (q_0, z_0 z_0')$
- ② Simulate all transitions done by M_2
 $\delta_1(q, a, z) = \delta_2(q, a, z)$
- ③ $\delta_1(q_1, \epsilon, z_0') = (q_f, \epsilon)$

↓
any state q_1
i/p completely
consumed i.e., ϵ
 z_0 is top of stack

↪ we can
go to final
state

Hence, proved

b) Design PDA for $L = \{a^i b^j c^k \mid k = j + i\}$. Illustrate the working for the string 'aabbcccc'.

$$\delta(q_0, a, z_0) = (q_0, a z_0)$$

$$\delta(q_0, a z_0) = (q_0, a a)$$

$$\delta(q_0, b, a) = (q_1, b a)$$

$$\delta(q_1, b, b) = (q_1, b b)$$

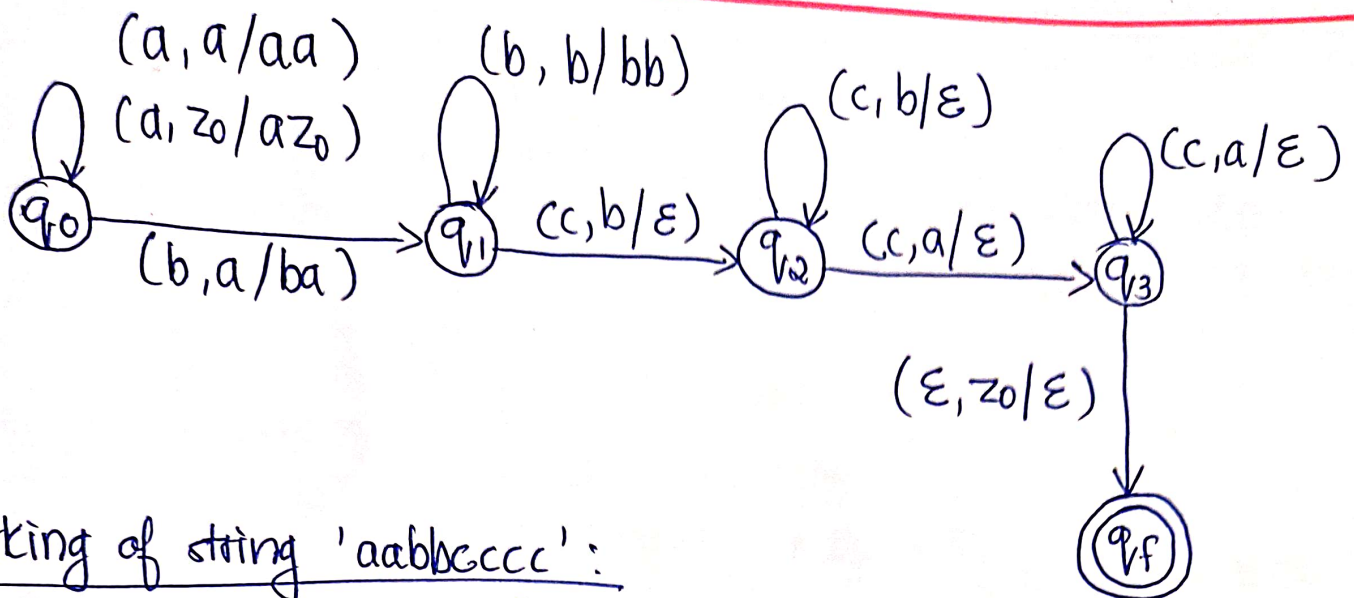
$$\delta(q_1, c, b) = (q_2, \epsilon)$$

$$\delta(q_2, c, b) = (q_2, \epsilon)$$

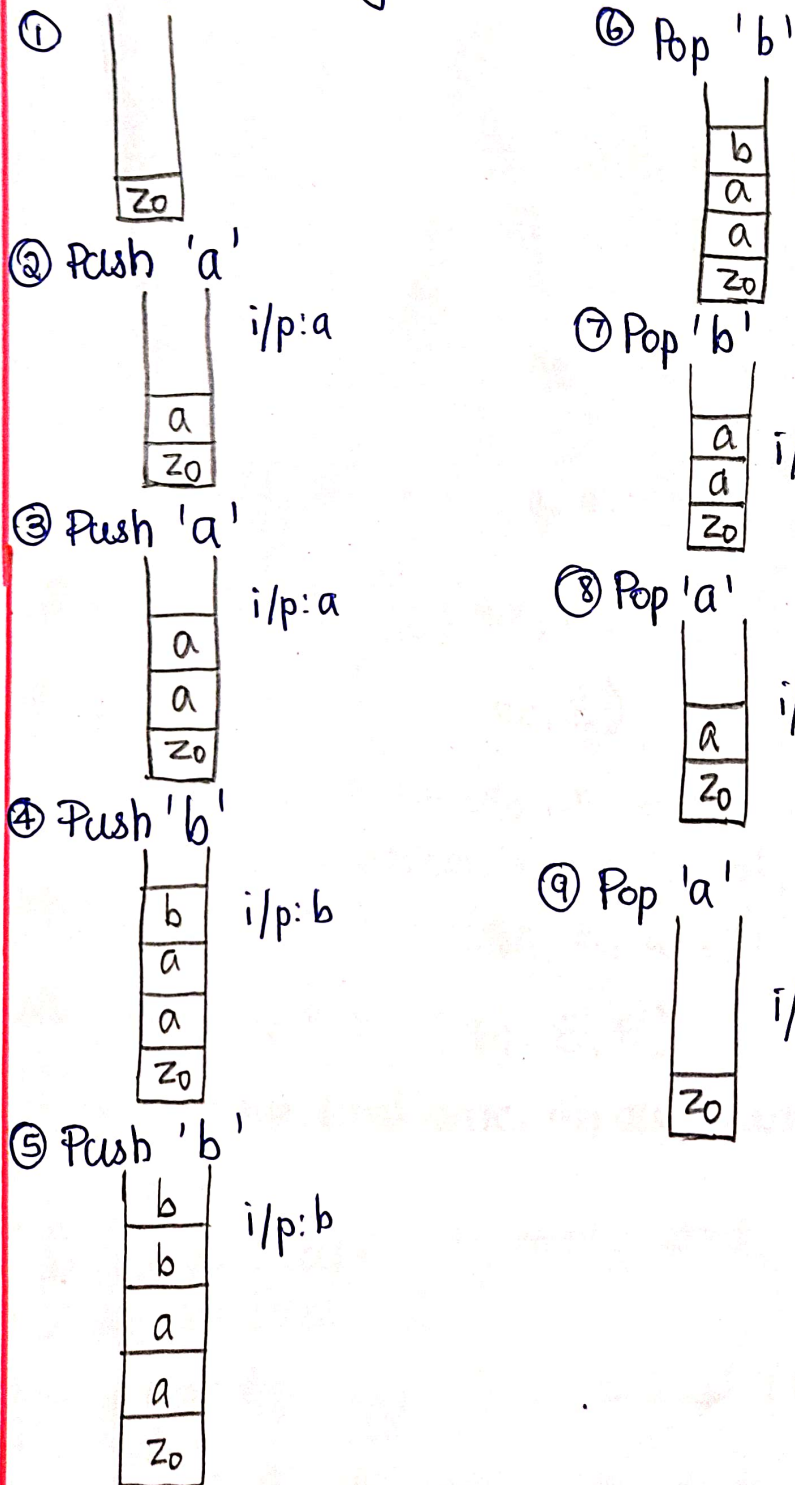
$$\delta(q_2, c, a) = (q_3, \epsilon)$$

$$\delta(q_3, c, a) = (q_3, \epsilon)$$

$$\delta(q_3, \epsilon, z_0) = (q_f, z_0)$$



Working of string 'aabbccccc':



$$(q_0, aabbccccc) = (q_0, az_0, aabbccccc)$$

$$(q_0, aabbccccc) =$$

$$aabbccccc \delta(q_0, a, z_0) = (q_0, az_0)$$

$$abbccccc \delta(q_0, a, a) = (q_0, aa)$$

$$bccccc \delta(q_0, b, a) = (q_1, ba)$$

$$bcccc \delta(q_1, b, b) = (q_1, bb)$$

$$cccc \delta(q_1, c, b) = (q_2, \epsilon)$$

$$ccc \delta(q_2, c, b) = (q_2, \epsilon)$$

$$cc \delta(q_2, c, a) = (q_3, \epsilon)$$

$$c \delta(q_3, c, a) = (q_3, \epsilon)$$

$$\delta(q_3, \epsilon, z_0) = q_f$$

Q a) Design PDA for $L = \{ww^R / w \in \{a, b\}^*\}$.

b) Stat and prove any three closure properties of Context free ~~gram~~ languages

Ans

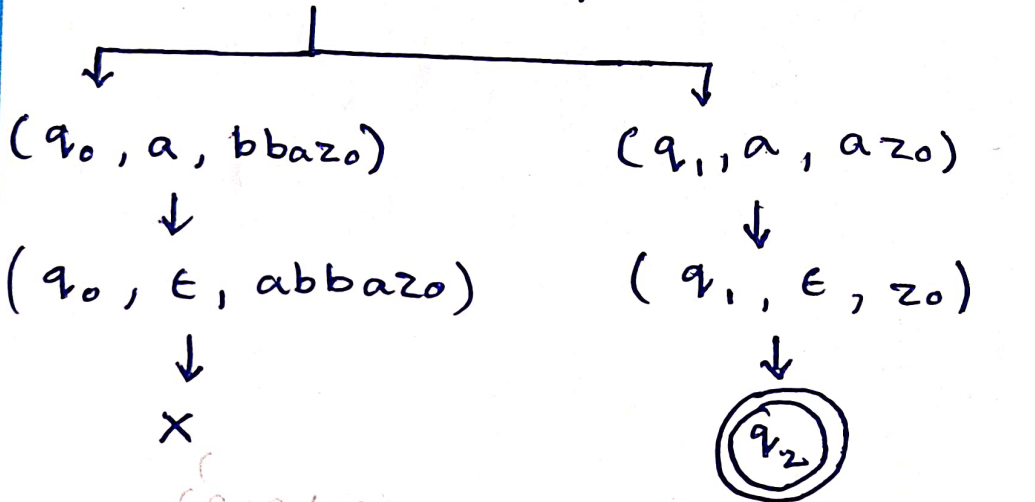
a) $\omega = \{aa, bb, abba, aabbaa, \dots\}$

$w = abba$

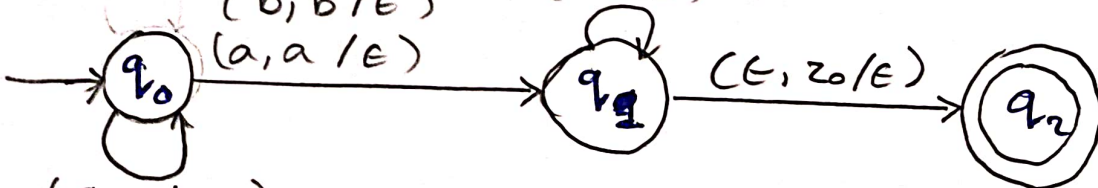
$$(q_0, abba, z_0)$$

↓
(q₀, bba, azo)

(q_0, b_a, b_{az_0})



$(b, a / \epsilon)$
 $(a, b / \epsilon)$
 $(b, b / \epsilon)$
 $(a, a / \epsilon)$

$$\begin{array}{l} (a, b/e) \quad (b, a/e) \\ (b, b/e) \\ (a, a/e) \end{array}$$


$(a, z_0 / az_0)$
 $(b, z_0 / bz_0)$
 $(a, a / aa)$
 $(b, b / bb)$
 $(a, b / ab) (b, a / ba)$

$$Q = \{ q_0, q_1, q_2 \}$$

$$\Sigma = \{ a, b \}$$

$$F = \{ z_0, a, b \}$$

$$q_0 = q_0$$

$$z_0 = z_0$$

$$F = q_2$$

considers the string $abba$

$$\delta(q_0, a, z_0) = (q_0, a z_0)$$

$$\delta(q_0, b, a) = (q_0, ba)$$

$$\delta(q_0, b, b) = (q_1, \epsilon)$$

$$\delta(q_1, \epsilon, \epsilon) = (q_1, \epsilon)$$

$$\delta(q_1, \epsilon, z_0) = (q_2, \epsilon)$$

b)

1) Union

if L_1 and L_2 are two context free languages their union will also be a context free language

Let $L_1 = \{a^n b^n / n > 0\}$ corresponding to a grammar G_1 will have a production

$$P \quad S_1 \rightarrow a S_1 b / a b$$

Let $L_2 = \{c^m d^m / m \geq 0\}$ corresponding to a grammar G_2 will have production $P \quad S_2 \rightarrow c S_2 d / \epsilon$

Union of L_1 and L_2 is $L = L_1 \cup L_2$

$$= \{a^n b^n \cup c^m d^m\}$$

corresponding to a grammar G will have an additional production

$$S \rightarrow S_1 / S_2$$

2) Concatenation

if L_1 and L_2 are two context free languages their concatenation $L_1.L_2$ will also be context free

Let $L_1 = \{a^n b^n / n > 0\}$ corresponding to a grammar G_1 will have a production

$$P \quad S_1 \rightarrow a S_1 b / a b$$

Let $L_2 = \{c^m d^m \mid m > 0\}$ corresponding to a grammar G_2 will have a production

$$p \quad s_2 \rightarrow c s_2 d \mid \epsilon$$

The concatenation of L_1 and L_2 i.e. $L = L_1 \cdot L_2$

$$L = L_1 L_2 = \{a^n b^n c^m d^m\}$$

The corresponding Grammar G will have an additional production

$$S \rightarrow S_1 S_2$$

3) Kleen Closure

if L is a context free language L^* is also a context free language

Let $L_1 = \{a^n b^n \mid n \geq 0\}$ corresponding to the grammar G_1 will have a production

$$s_1 \rightarrow a s_1 b \mid \epsilon$$

$$\text{Kleen Closure } L^* = \{a^n b^n\}^*$$

The corresponding grammar G will have an additional production

$$s_1 \rightarrow s s_1 \text{ or } \epsilon$$

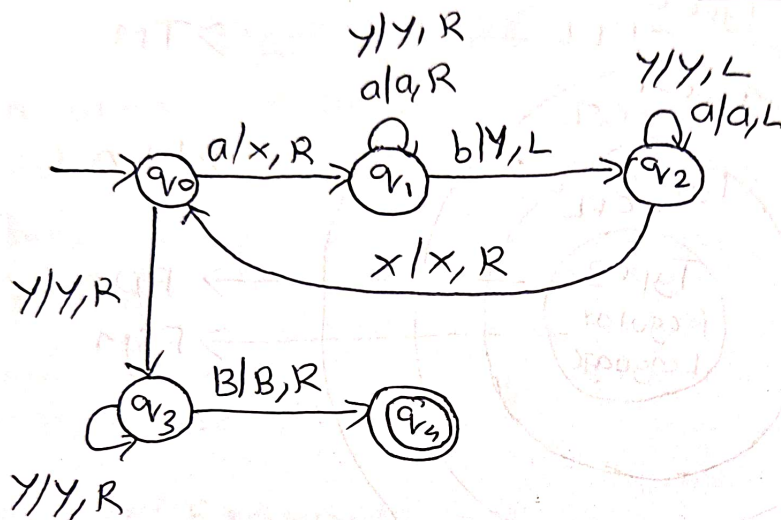
19) Qn)
(a)

Design TM for $L = \{a^n b^n \mid n > 0\}$. Trace out the working for the input string 'aabb'

Ans)

The string that we have to consider is 'aabb'

B a a b b B



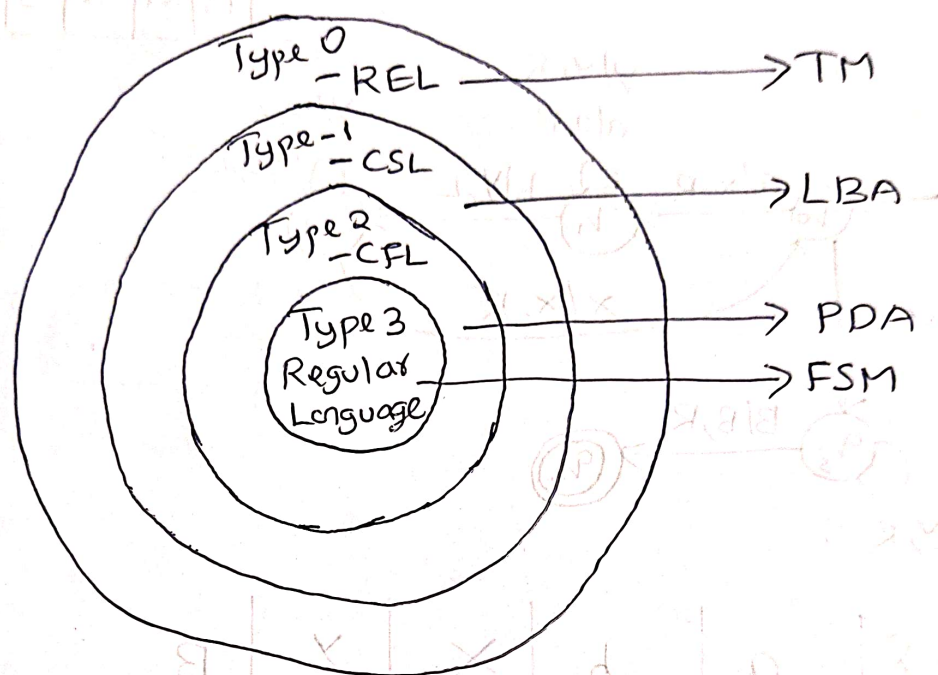
δ	a	b	x	y	B
q_0	q_1	—	—	q_3	—
q_1	q_1	q_2	—	q_1	—
q_2	q_2	—	q_0	q_2	—
q_3	—	—	—	q_3	q_4
q_4	—	—	—	—	—

19) Qn)
(b)

Explain Chomsky hierarchy for formal languages and evaluate various types.

Ans)

Chomsky Classification of Formal Languages



→ Chomsky has classified all grammar in 4 categories (Type 0 - Type 3) based on the right hand side forms of the product.

→ Type 3 :- Regular languages :- Finite Automata
Only one terminal at the RHS and RHS contains ϵ or terminal followed by a non-terminal or a non-terminal followed by a terminal

eg:- $A \rightarrow \epsilon, A \rightarrow a, A \rightarrow aB, A \rightarrow Ba$

→ Type 2: Context Free Grammar - Push Down Automata

A production is of the form $A \rightarrow \alpha$ where A is a non-terminal, α is any terminal or

eg:

non-terminal.

$A \rightarrow \alpha$

$S \rightarrow aSa$

$S \rightarrow S+S$

→

Type 1: Context Sensitive Language: Linear Bounded Automata

The production of the form $\alpha \rightarrow \beta$ in which α and β may have both terminals and non-terminals combination

$$|\alpha| \leq |\beta|$$

$$\beta \neq \epsilon$$

eg:

$aB \rightarrow aaA$

$aB \rightarrow aa$

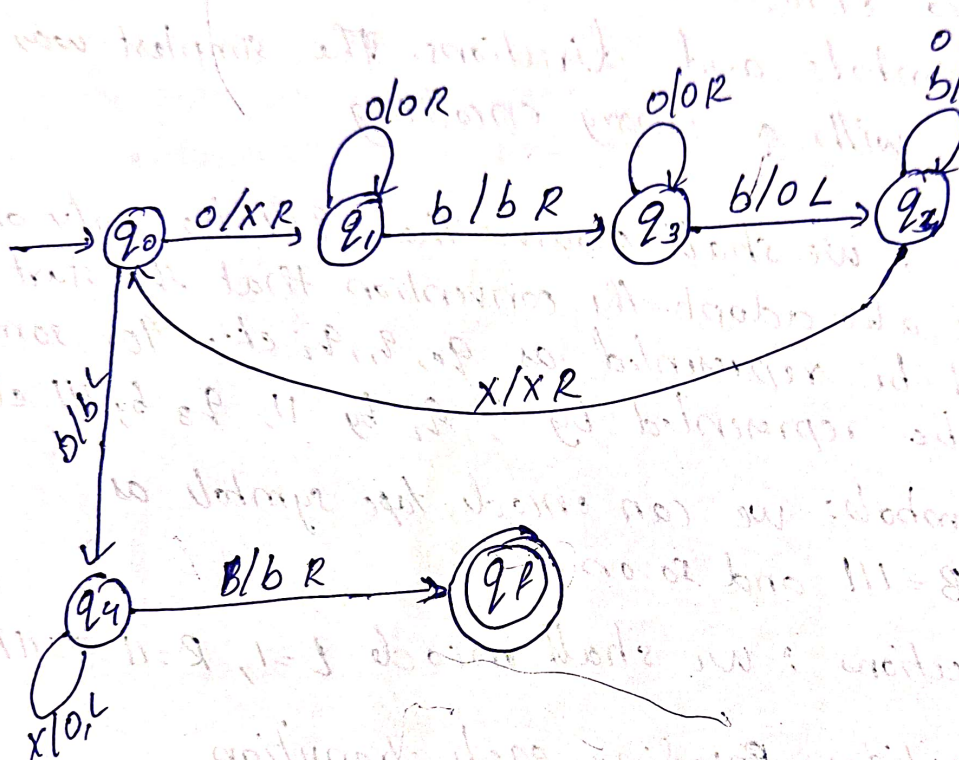
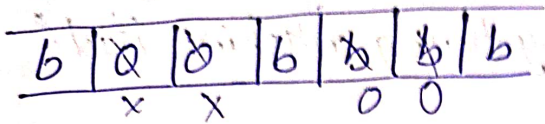
→

Type 0: Recursively enumerable languages-TM

• Accept all type of grammar, free from restrictions.

$$\text{Type } 0 \leq \text{Type } 1 \leq \text{Type } 2 \leq \text{Type } 3$$

20a) Design a TM to copy a block of zeros to the right side, leaving one blank symbol (b) in between. Assume that initially the input tape contains $b0^n b$ and TM halts with $b0^n b0^n b$ as the tape content.



20b) Explain the working of Universal Turing Machine with its encoding scheme.

A Turing machine is said to be Universal Turing Machine if it can accept.

- The Input data;
- An Algorithm (description) for computing.

This is precisely what a general purpose digital computer does. A digital computer accepts a program

written in high level languages.

- Thus a general purpose Turing machine will be called a universal Turing machine if it is powerful enough to simulate the behaviour of any digital computer, including any Turing machine itself.

* Encoding Scheme :-

One way to encode a TM is to encode all of its states, tape symbols and directions. The simplest way to do this is with a unary encoding.

- Encoding states : we shall assume that TM has only one final state. we also adopt the convention that the start and final states will be represented as q_0, q_1, q_i etc... For some i , then q_i will be represented by 1 , q_2 by 11 , q_3 by 111 etc...
- Encoding Tape symbols: we can encode tape symbols as $a = 1$, $b = 11$, $B = 111$ and so on.
- Encoding Directions : we shall encode $L = 1$, $R = 11$, $S = 111$.
- Encoding transitions: Encoding each transition $\delta(q_i, a_j) = (q_k, a_j, D)$ as a sequence of 5 codes : q_i, a_j, q_k and a_j , and D . if we encode 5 tuple $(q_0, 1, q_1, 0, L)$ as 1011011101010101 we could use the sequence 10101010101010101 .
- Encoding Turing Machine : We can encode a TM as a binary string consisting of all encoded transitions separated by 00 .